Overview of CycleGAN

Subject: CycleGAN

1.

Vanishing gradient Conversely, if the discriminator learns too fast compared to the generator, then the discriminator could almost perfectly distinguish μ G θ , μ ref {\displaystyle \mu _{G_{\theta }},\mu _{\text{ref}}} . In such case, the generator G θ {\displaystyle G_{\theta }} could be stuck with a very high loss no matter which direction it changes its θ {\displaystyle \theta } , meaning that the gradient ∇ θ L (G θ , D ζ) {\displaystyle \nabla _{\theta }L(G_{\theta },D_{\zeta })} would be close to zero. In such case, the generator cannot learn, a case of the vanishing gradient problem. Intuitively speaking, the discriminator is too good, and since the generator cannot take any small step (only small steps are considered in gradient descent) to improve its payoff, it does not even try. One important method for solving this problem is the Wasserstein GAN.

2.

Compared to fully visible belief networks such as WaveNet and PixelRNN and autoregressive models in general, GANs can generate one complete sample in one pass, rather than multiple passes through the network. Compared to Boltzmann machines and linear ICA, there is no restriction on the type of function used by the network. Since neural networks are universal approximators, GANs are asymptotically consistent. Variational autoencoders might be universal approximators, but it is not proven as of 2017.

3.

The generator's task is to approach μ G ≈ μ ref {\displaystyle \mu _{G}\approx \mu _{\text{ref}}} , that is, to match its own output distribution as closely as possible to the reference distribution. The discriminator's task is to output a value close to 1 when the input appears to be from the reference distribution, and to output a value close to 0 when the input looks like it came from the generator distribution.

4.

In practice The generative network generates candidates while the discriminative network evaluates them. This creates a contest based on data distributions, where the generator learns to map from a latent space to the true data distribution, aiming to produce candidates that the discriminator cannot distinguish from real data. The discriminator's goal is to correctly identify these candidates, but as the generator improves, its task becomes more challenging, increasing the discriminator's error rate. A known dataset serves as the initial training data for the discriminator. Training involves presenting it with samples from the training dataset until it achieves acceptable accuracy. The generator is trained based on whether it succeeds in fooling the discriminator. Typically, the generator is seeded with randomized input that is sampled from a predefined latent space (e.g. a multivariate normal distribution). Thereafter, candidates synthesized by the generator are evaluated by the discriminator. Independent backpropagation procedures are applied to both networks so that the generator produces better samples, while the discriminator becomes more skilled at flagging synthetic samples. When used for image generation, the generator is typically a deconvolutional neural network, and the discriminator

is a convolutional neural network.

5.

Conditional GAN Conditional GANs are similar to standard GANs except they allow the model to conditionally generate samples based on additional information. For example, if we want to generate a cat face given a dog picture, we could use a conditional GAN. The generator in a GAN game generates μ G {\displaystyle \mu _{G}} , a probability distribution on the probability space Ω {\displaystyle \Omega } . This leads to the idea of a conditional GAN, where instead of generating one probability distribution on Ω {\displaystyle \Omega } , the generator generates a different probability distribution μ G (c) {\displaystyle \mu _{G}(c)} on Ω {\displaystyle \Omega } , for each given class label c {\displaystyle c} . For example, for generating images that look like ImageNet, the generator should be able to generate a picture of cat when given the class label "cat". In the original paper, the authors noted that GAN can be trivially extended to conditional GAN by providing the labels to both the generator and the discriminator. Concretely, the conditional GAN game is just the GAN game with class labels provided: L (μ G , D) := E c ~ μ C , x ~ μ ref (c) ■ [ln ■ D (x , c)] + E c ~ μ C , x ~ μ G (c) ■ [ln ■ (1 − D (x , c))] {\displaystyle L(\mu _{G},D):=\operatorname {E} _{c\sim \mu _{C},x\sim \mu _{\text{ref}}(c)}[\ln D(x,c)]+\operatorname {E} _{c\sim \mu _{C},x\sim \mu _{G}(c)}[\ln(1-D(x,c))]} where μ C {\displaystyle \mu _{C}} is a probability distribution over classes, μ ref (c) {\displaystyle \mu _{\text{ref}}(c)} is the probability distribution of real images of class c {\displaystyle c} , and μ G (c) {\displaystyle \mu _{G}(c)} the probability distribution of images generated by the generator when given class label c {\displaystyle c} . In 2017, a conditional GAN learned to generate 1000 image classes of ImageNet.

6.

Invertible data augmentation When there is insufficient training data, the reference distribution μ ref {\displaystyle \mu _{\text{ref}}} cannot be well-approximated by the empirical distribution given by the training dataset. In such cases, data augmentation can be applied, to allow training GAN on smaller datasets. Naïve data augmentation, however, brings its problems. Consider the original GAN game, slightly reformulated as follows: { min D L D (D , μ G) = − E x ~ μ ref ■ [ln ■ D (x)] − E x ~ μ G ■ [ln ■ (1 − D (x))] min G L G (D , μ G) = − E x ~ μ G ■ [ln ■ (1 − D (x))] {\displaystyle {\begin{cases}\min _{D}L_{D}(D,\mu _{G})=-\operatorname {E} _{x\sim \mu _{\text{ref}}}[\ln D(x)]-\operatorname {E} _{x\sim \mu _{G}}[\ln(1-D(x))]\\\min _{G}L_{G}(D,\mu _{G})=-\operatorname {E} _{x\sim \mu _{G}}[\ln(1-D(x))]\end{cases}}} Now we use data augmentation by randomly sampling semantic-preserving transforms T : Ω → Ω {\displaystyle T:\Omega \to \Omega } and applying them to the dataset, to obtain the reformulated GAN game: { min D L D (D , μ G) = − E x ~ μ ref , T ~ μ trans ■ [ln ■ D (T (x))] − E x ~ μ G ■ [ln ■ (1 − D (x))] min G L G (D , μ G) = − E x ~ μ G ■ [ln ■ (1 − D (x))] {\displaystyle {\begin{cases}\min _{D}L_{D}(D,\mu _{G})=-\operatorname {E} _{x\sim \mu _{\text{ref}},T\sim \mu _{\text{trans}}}[\ln D(T(x))]-\operatorname {E} _{x\sim \mu _{G}}[\ln(1-D(x))]\\\min _{G}L_{G}(D,\mu _{G})=-\operatorname {E} _{x\sim \mu _{G}}[\ln(1-D(x))]\end{cases}}} This is equivalent to a GAN game with a different distribution μ ref ′ {\displaystyle \mu _{\text{ref}}'} , sampled by T (x) {\displaystyle T(x)} , with x ~ μ ref , T ~ μ trans {\displaystyle x\sim \mu _{\text{ref}},T\sim \mu _{\text{trans}}} . For example, if μ ref {\displaystyle \mu _{\text{ref}}} is the distribution of images in ImageNet, and μ trans {\displaystyle \mu _{\text{trans}}} samples identity-transform with probability 0.5, and horizontal-reflection with probability 0.5, then μ ref ′

{\displaystyle \mu _{\text{ref}}'} is the distribution of images in ImageNet and horizontally-reflected ImageNet, combined. The result of such training would be a generator that mimics μ ref ′ {\displaystyle \mu _{\text{ref}}'} . For example, it would generate images that look like they are randomly cropped, if the data augmentation uses random cropping. The solution is to apply data augmentation to both generated and real images: { min D L D (D , μ G) = − E x ~ μ ref , T ~ μ trans ■ [ln ■ D (T (x))] − E x ~ μ G , T ~ μ trans ■ [ln ■ (1 − D (T (x)))] min G L G (D , μ G) = − E x ~ μ G , T ~ μ trans ■ [ln ■ (1 − D (T (x)))] {\displaystyle {\begin{cases}\min _{D}L_{D}(D,\mu _{G})=-\operatorname {E} _{x\sim \mu _{\text{ref}},T\sim \mu _{\text{trans}}}[\ln D(T(x))]-\operatorname {E} _{x\sim \mu _{G},T\sim \mu _{\text{trans}}}[\ln(1-D(T(x)))]\\\min _{G}L_{G}(D,\mu _{G})=-\operatorname {E} _{x\sim \mu _{G},T\sim \mu _{\text{trans}}}[\ln(1-D(T(x)))]\end{cases}}} The authors demonstrated high-quality generation using just 100-picture-large datasets. The StyleGAN-2-ADA paper points out a further point on data augmentation: it must be invertible. Continue with the example of generating ImageNet pictures. If the data augmentation is "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", then there is no way for the generator to know which is the true orientation: Consider two generators G , G ′ {\displaystyle G,G'} , such that for any latent z {\displaystyle z} , the generated image G (z) {\displaystyle G(z)} is a 90-degree rotation of G ′ (z) {\displaystyle G'(z)} . They would have exactly the same expected loss, and so neither is preferred over the other. The solution is to only use invertible data augmentation: instead of "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", use "randomly rotate the picture by 90, 180, 270 degrees with 0.1 probability, and keep the picture as it is with 0.7 probability". This way, the generator is still rewarded to keep images oriented the same way as un-augmented ImageNet pictures. Abstractly, the effect of randomly sampling transformations T : Ω → Ω {\displaystyle T:\Omega \to \Omega } from the distribution μ trans {\displaystyle \mu _{\text{trans}}} is to define a Markov kernel K trans : Ω → P (Ω) {\displaystyle K_{\text{trans}}:\Omega \to {\mathcal {P}}(\Omega)} . Then, the data-augmented GAN game pushes the generator to find some μ ^ G ∈ P (Ω) {\displaystyle {\hat {\mu }}_{G}\in {\mathcal {P}}(\Omega)} , such that K trans ∗ μ ref = K trans ∗ μ ^ G {\displaystyle K_{\text{trans}}*\mu _{\text{ref}}=K_{\text{trans}}*{\hat {\mu }}_{G}} where ∗ {\displaystyle *} is the Markov kernel convolution. A data-augmentation method is defined to be invertible if its Markov kernel K trans {\displaystyle K_{\text{trans}}} satisfies K trans ∗ μ = K trans ∗ μ ′ ■ μ = μ ′ ∀ μ , μ ′ ∈ P (Ω) {\displaystyle K_{\text{trans}}*\mu =K_{\text{trans}}*\mu '\implies \mu =\mu '\quad \forall \mu ,\mu '\in {\mathcal {P}}(\Omega)} Immediately by definition, we see that composing multiple invertible data-augmentation methods results in yet another invertible method. Also by definition, if the data-augmentation method is invertible, then using it in a GAN game does not change the optimal strategy μ ^ G {\displaystyle {\hat {\mu }}_{G}} for the generator, which is still μ ref {\displaystyle \mu _{\text{ref}}} . There are two prototypical examples of invertible Markov kernels: Discrete case: Invertible stochastic matrices, when Ω {\displaystyle \Omega } is finite. For example, if Ω = { ↑ , ↓ , ← , → } {\displaystyle \Omega =\{\uparrow ,\downarrow ,\leftarrow ,\rightarrow \}} is the set of four images of an arrow, pointing in 4 directions, and the data augmentation is "randomly rotate the picture by 90, 180, 270 degrees with probability p {\displaystyle p} , and keep the picture as it is with probability (1 − 3 p) {\displaystyle (1-3p)} ", then the Markov kernel K trans {\displaystyle K_{\text{trans}}} can be represented as a stochastic matrix: [K trans] = [(1 − 3 p) p p p p (1 − 3 p) p p p p (1 − 3 p) p p p p (1 − 3 p)] {\displaystyle [K_{\text{trans}}]={\begin{bmatrix}(1-3p)&p&p&p\\p&(1-3p)&p&p\\p&p&(1-3p)&p\\p&p&p&(1-3p)\end{bmatrix}}} and K trans {\displaystyle K_{\text{trans}}} is an invertible kernel iff [K trans] {\displaystyle [K_{\text{trans}}]} is an

invertible matrix, that is, p ≠ 1 / 4 {\displaystyle p\neq 1/4} . Continuous case: The gaussian kernel, when Ω = R n {\displaystyle \Omega =\mathbb {R} ^{n}} for some n ≥ 1 {\displaystyle n\geq 1} . For example, if Ω = R 256 2 {\displaystyle \Omega =\mathbb {R} ^{256^{2}}} is the space of 256x256 images, and the data-augmentation method is "generate a gaussian noise z ~ N (0 , I 256 2) {\displaystyle z\sim {\mathcal {N}}(0,I_{256^{2}})} , then add ■ z {\displaystyle \epsilon z} to the image", then K trans {\displaystyle K_{\text{trans}}} is just convolution by the density function of N (0 , ■ 2 I 256 2) {\displaystyle {\mathcal {N}}(0,\epsilon ^{2}I_{256^{2}})} . This is invertible, because convolution by a gaussian is just convolution by the heat kernel, so given any μ ∈ P (R n) {\displaystyle \mu \in {\mathcal {P}}(\mathbb {R} ^{n})} , the convolved distribution K trans ∗ μ {\displaystyle K_{\text{trans}}*\mu } can be obtained by heating up R n {\displaystyle \mathbb {R} ^{n}} precisely according to μ {\displaystyle \mu } , then wait for time ■ 2 / 4 {\displaystyle \epsilon ^{2}/4} . With that, we can recover μ {\displaystyle \mu } by running the heat equation backwards in time for ■ 2 / 4 {\displaystyle \epsilon ^{2}/4} . More examples of invertible data augmentations are found in the paper.